INTEGRATIVE PROGRAMMING AND TECHNOLOGIES

SYSTEM INTEGRATION AND ARCHITECTURE

Final Project

STOCKWISE: INVENTORY AND PURCHASING SYSTEM

A MERN Stack Web Application for Santos Family Hardware, a Family-Owned Hardware Store

Technical Documentation

Group Members

Ivan Tadena (Developer)

Subjects

Integrative Programming and Technologies (INF238)

System Integration and Architecture (CTSYSINL)

Date: October 15, 2026

Live application: https://stockwise-ivan.vercel.app

Source code: https://github.com/Ivan259483/stockwise

Table of Contents

Project Overview 3
Objectives 6
Scope and Limitations 7
System Architecture 8
Database Design 12
User Interface Design 22
Feature List 38
Summary and Future Work 42
Sources 44

Project Overview

Background and Problem

Micro, small and medium enterprises (MSMEs) account for more than 99 percent of all business establishments in the Philippines, according to the Department of Trade and Industry. Many of them, such as neighborhood hardware stores, still record their stock in paper logbooks and their supplier orders in a notebook or in text messages.

Manual record keeping causes recurring problems. Owners discover that an item has run out only when a customer asks for it, they reorder items that are already plentiful, and nobody can tell who changed a quantity or whether a supplier delivery was really received. Purchases are agreed by phone without a written approval, so the owner cannot answer "What did we order this month, from whom, and did it arrive?"

Target Client: Santos Family Hardware

The sample organization for this project is **Santos Family Hardware**, a family-owned hardware store run by the spouses Roberto and Maricel Santos with eight employees. The store sells about 35 kinds of products in eight categories (hand tools, power tools, electrical, plumbing, paints and coatings, construction materials, fasteners and hardware, and safety gear) to walk-in customers and small contractors, and it buys from six regular suppliers in Luzon and Cebu. The store and its people are fictional; the name is shown as text only, without a logo, and every demo account uses a made-up email domain that does not exist.

Users and Roles

Table 1. *Users of the system at Santos Family Hardware*

Person (job)	Role	What they do in StockWise
Roberto and Maricel Santos (owners)	Admin	Manage the catalog, suppliers and user accounts; approve or reject purchase orders; archive and restore records.
Lorna Mercado (purchasing clerk)	Staff	Watches low stock and creates purchase order requests.

Person (job)	Role	What they do in StockWise
Jomar Dizon, Rodel Bautista (warehouse)	Staff	Record damaged stock, request urgent orders and receive deliveries.
Ferdinand Garcia (receiving)	Staff	Checks deliveries and marks approved purchase orders as received.
Kristine Mae Villanueva, Angelica Ramos (cashiers); Paolo Navarro, Ivan Tadena (sales)	Staff	Record sales as stock-out movements and check stock levels for customers.

Business Process: Purchase Orders

The core business process added for System Integration and Architecture is buying stock from suppliers with a **purchase order (PO)**. Figure 1 shows the workflow. When an item reaches its reorder level, the dashboard lists it and the admins receive an email alert. A staff member creates a PO request with the supplier, items, quantities and unit costs, and the PO starts as **Requested**. An admin then **approves** it, or **rejects** it with a reason. When the delivery arrives, a staff member or an admin marks the approved PO as **Received**, which is the completed state: the system records one stock-in movement per item and updates the stock with the same atomic update used by ordinary stock movements. A request can be **cancelled** while it is still Requested. Every step is written to the PO's timeline with who did it and when, and closed POs can be archived.

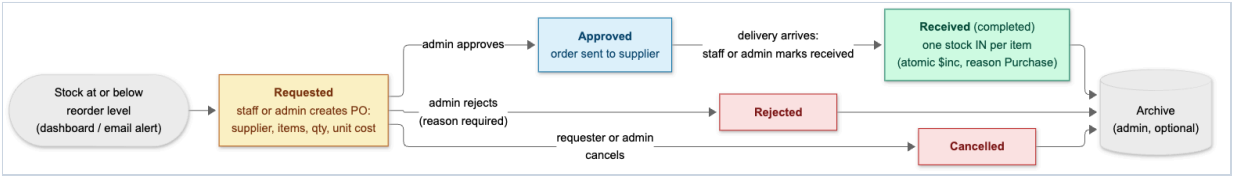


Figure 1. *Purchase order workflow at Santos Family Hardware*

Proposed Solution

StockWise is a web-based inventory and purchasing system built on the MERN stack (MongoDB, Express, React and Node.js). It replaces the logbook with a searchable product

catalog and records every stock change as a *stock movement* that stores who made the change, when, why, and the quantity before and after it. Purchase orders connect the store to its suppliers from request to delivery. Users manage their own profile and can recover a forgotten password with a six-digit code sent by email, admins manage user accounts, and records are archived instead of deleted so that the history always stays complete.

The system integrates several services: a React client and an Express REST API deployed as two Vercel projects, a MongoDB Atlas database, and Gmail SMTP for password-recovery codes and low-stock alerts. It runs in any modern browser on a desktop computer or a mobile phone.

Live Deliverables

Table 2. *Deployed resources and demo accounts*

Resource	URL or credentials
Web application	https://stockwise-ivan.vercel.app
REST API	https://stockwise-api-ivan.vercel.app
Source code	https://github.com/Ivan259483/stockwise
Demo admin (owner)	roberto.santos@santosfamilyhw-demo.com / Admin123!
Demo staff (cashier)	kristine.villanueva@santosfamilyhw-demo.com / Staff123!

Objectives

The general objective is to develop a secure, integrated web application that lets a family-owned hardware store track its inventory accurately and buy stock from suppliers through a controlled, auditable process. The specific, measurable objectives are the following:

- **Secure access:** require a valid JSON Web Token (JWT) for every endpoint except health, login, registration and password recovery, and enforce two roles (admin and staff) on both the server and the user interface.
- **Account management:** let every user view and edit their name and email and change their password, and let a user who forgot their password reset it with a six-digit emailed code that is stored hashed, expires in 15 minutes and is rate-limited.
- **User management:** let admins add users, edit their name, email and role, activate or deactivate them, and archive them, while preventing admins from demoting, deactivating or archiving themselves.
- **Purchase order process:** support the Requested, Approved, Rejected, Received and Cancelled states, with a timeline of who did what and when, and record one stock-in movement per item when a PO is received.
- **Archive instead of delete:** keep products, suppliers, users and closed purchase orders forever, hide archived records from every active list, search and form, and let admins view and restore them.

- **Full audit trail and stock integrity:** record every quantity change as a stock movement and never allow negative stock, using atomic database updates that are safe under concurrent requests.
- **Integration with email:** send password-recovery codes and low-stock alerts through Gmail SMTP, once per product until it is restocked.
- **Verified quality:** pass an automated 115-check API smoke test, a 49-step browser end-to-end test and a data check of the presentation data, all on isolated databases.

Scope and Limitations

Scope

- Registration and login with JWT authentication, bcrypt password hashing and two roles (admin and staff).
- Account management: a profile page and password recovery by emailed code.
- User management by admins: add, edit (name, email, role), activate or deactivate, and archive users.
- Management of products, categories and suppliers, including an optional product image of up to 1 MB.
- Stock-in and stock-out recording with reasons and a filterable movement history.
- Purchase orders from request to approval or rejection, cancellation and receipt into stock, with status filters, a details page with a timeline, and counts on the dashboard.
- An Archive module for products, suppliers, users and closed purchase orders, with view and restore.
- Email integration through Gmail SMTP for password-recovery codes and low-stock alerts.
- A dashboard with statistics, charts, a restock list, recent movements and purchase order counts.
- Input validation and error handling on both the client and the server, and a responsive interface for desktop, tablet and mobile browsers.

Limitations

- The system manages a single store; it does not support multiple branches or transfers between locations.

- It is not a point-of-sale (POS) system. Sales are recorded as stock-out movements, without receipts or payments.
- A purchase order is received in full in one step; partial deliveries and supplier invoices are not tracked, and receiving does not change the product's cost price.
- Products are identified by SKU text only; barcode scanning is not supported, and quantities are whole numbers.
- Email depends on a Gmail account and its daily sending limit. Demo accounts use a domain that does not exist, so the system never emails them; their alerts appear in the server log or go to an optional extra address (STOCK_ALERT_CC).
- Categories are still deleted rather than archived; a category that is used by any product, including archived ones, cannot be deleted.
- MongoDB Atlas on the free tier is used without multi-document transactions; receiving a PO uses atomic single-document updates with compensation if a step fails.
- The free hosting tiers may add a short delay after a period of inactivity (a serverless "cold start"), and rate limits are kept in memory per serverless instance.

System Architecture

StockWise follows a three-tier architecture with one external service. The presentation tier (React) and the application tier (Express) are deployed as two separate Vercel projects and communicate only through a RESTful JSON API over HTTPS. The data tier is a MongoDB database on MongoDB Atlas, and the API sends email through Gmail SMTP. Figure 2 shows the tiers, the request pipeline inside the API and the integrations.

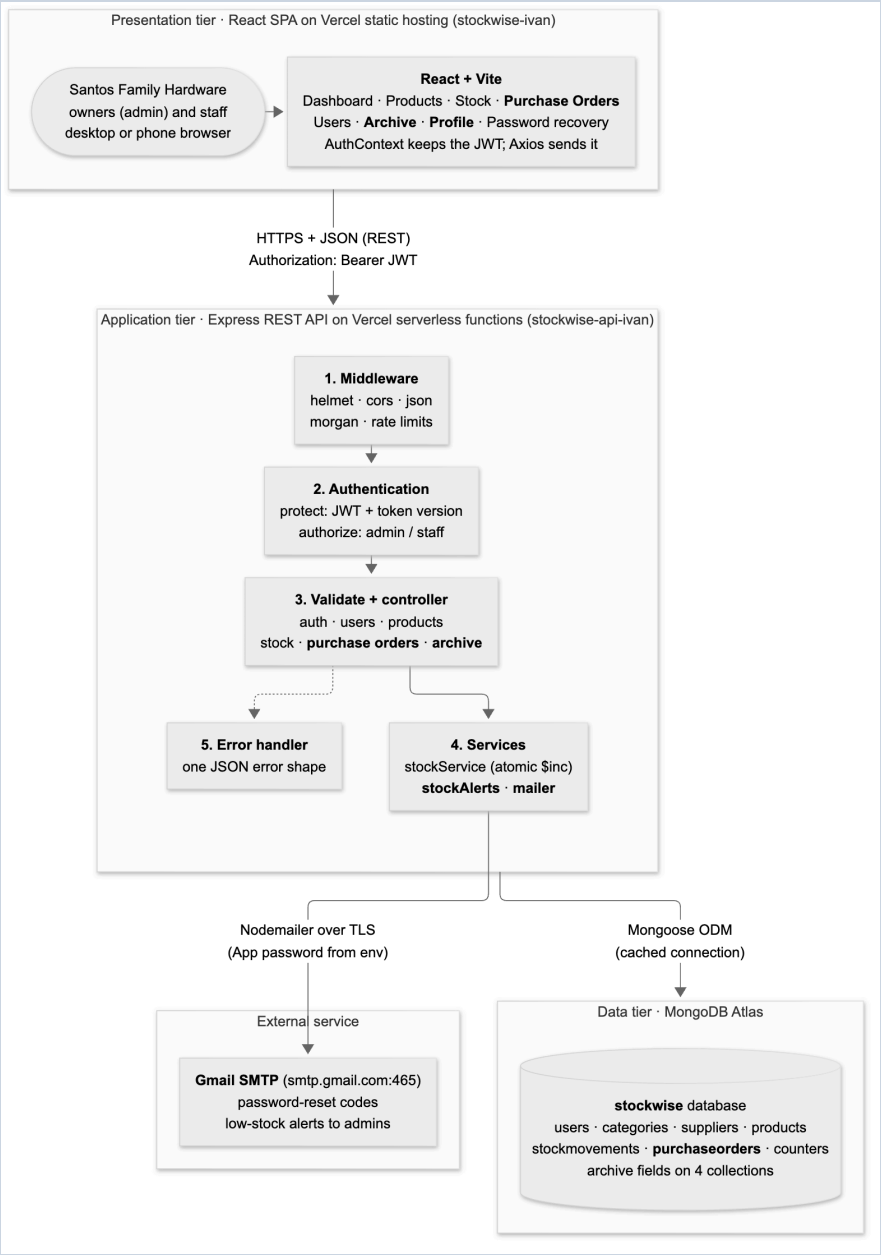


Figure 2. *System architecture of StockWise with Gmail SMTP and purchase orders*

Presentation Tier (Client)

The client is a single-page application built with React 19 and Vite and styled with Tailwind CSS. React Router maps each URL to a page, and route guards (ProtectedRoute, AdminRoute and PublicOnlyRoute) keep signed-out users and staff away from pages they may not use; the API enforces the same rules. All server calls go through one Axios instance whose interceptors attach the JWT, show a notification for every failed request and end the session when the server

answers 401. The store name, Santos Family Hardware, appears as text in the header and on the login page.

Application Tier (REST API)

The API is an Express 5 application organized into models, controllers, routes, middleware, validators, services and utilities. Every request passes through the same pipeline:

1. **Security and parsing middleware:** helmet, cors (only the deployed client and localhost), express.json and morgan; rate limits protect login, registration and password recovery.
2. **Authentication and authorization:** protect verifies the JWT, reloads the user and rejects deactivated or archived users and tokens issued before the last password change; authorize checks the role.
3. **Validation and controller:** express-validator rules check the request, and the controller performs the business logic using Mongoose models.
4. **Services:** stockService performs every quantity change atomically and writes the movement; stockAlerts and mailer send email through Gmail SMTP.
5. **Central error handler:** every error becomes the same JSON shape with an appropriate HTTP status code.

Data Tier (Database)

MongoDB Atlas stores seven collections in the stockwise database: users, categories, suppliers, products, stockmovements, purchaseorders and counters. Development and automated tests use separate local databases (stockwise_dev and stockwise_test), so tests never touch the demo or live data.

Integration with Gmail SMTP

The API sends email with Nodemailer over TLS to smtp.gmail.com on port 465, authenticated with a Google App password kept only in environment variables. Two features use it:

1. **Password recovery:** POST /api/auth/forgot-password generates a random six-digit code, stores only its bcrypt hash with a 15-minute expiry, and emails the code. The answer is the same whether or not the email exists, a new code is issued at most once a minute per account, and requests are rate-limited per IP address. POST /api/auth/reset-password

accepts the code at most five times; a correct code sets the new password and signs out every existing session.

2. **Low-stock alerts:** when a stock-out leaves a product at or below its reorder level, the API atomically marks the product as alerted and emails every active admin. Restocking above the reorder level clears the mark, so each product sends one alert per low period. An email failure never fails the sale.

Email is never sent to test domains or to the made-up demo domain. In development, if the Gmail credentials are not configured, the message (including a reset code) is printed in the backend console instead; in production only a warning is logged.

REST and JWT Authentication Flow

1. The client sends POST /api/auth/login with the email address and password.
2. The API compares the password with the stored bcrypt hash, refuses deactivated and archived accounts, and returns a JWT that expires after one day. The token carries the user id and a token version.
3. The client stores the token and adds "Authorization: Bearer <token>" to every later request.
4. For each protected request the API verifies the token, reloads the user, compares the token version (it changes when the password changes) and checks the role. It answers { success: true, data } or { success: false, message, errors }.
5. A missing, invalid, expired or outdated token returns 401; the client clears the session and shows the login page.

Stock Integrity and Receiving

A stock-out is one atomic findOneAndUpdate that decreases the quantity only if enough stock is available; otherwise the API returns 409 "Insufficient stock: only X available". Stock-ins use the atomic $inc operator. Each change then writes a stock movement, and if that write fails the quantity change is reversed.

Purchase order status changes use the same idea: each transition is a single update that matches only while the PO is still in the expected status, so two admins cannot approve the same PO twice and a delivery cannot be received twice. Receiving first claims the Received status, then adds each item to stock through the shared stock service with reason Purchase and

a link to the PO. If any item fails, the items already added are reverted and the PO returns to Approved.

Archive Design

Products, suppliers, users and purchase orders share an archive plugin that adds archived, archivedAt, archivedBy and archiveReason fields and an archive log. Every active query filters with { archived: { $ne: true } }, so records created before the feature existed count as active without a data migration. Archiving is refused for a product or supplier that is on an open (Requested or Approved) purchase order, for a PO that is not closed, and for the admin's own account. Archived users cannot log in. Nothing is hard-deleted, so stock movements and purchase orders always keep their references.

Database Design

Figures 3 to 5 show the collections and their relationships. Figure 3 covers the product catalog, Figure 4 the stock history and Figure 5 the purchase orders, whose items and timeline events are embedded in the PO document. Stock movements and PO items store copies of the product name and SKU so that the history stays readable after a product is renamed or archived. All collections have automatic createdAt and updatedAt timestamps, which are omitted from the diagrams. Field constraints are listed in Tables 3 to 11.

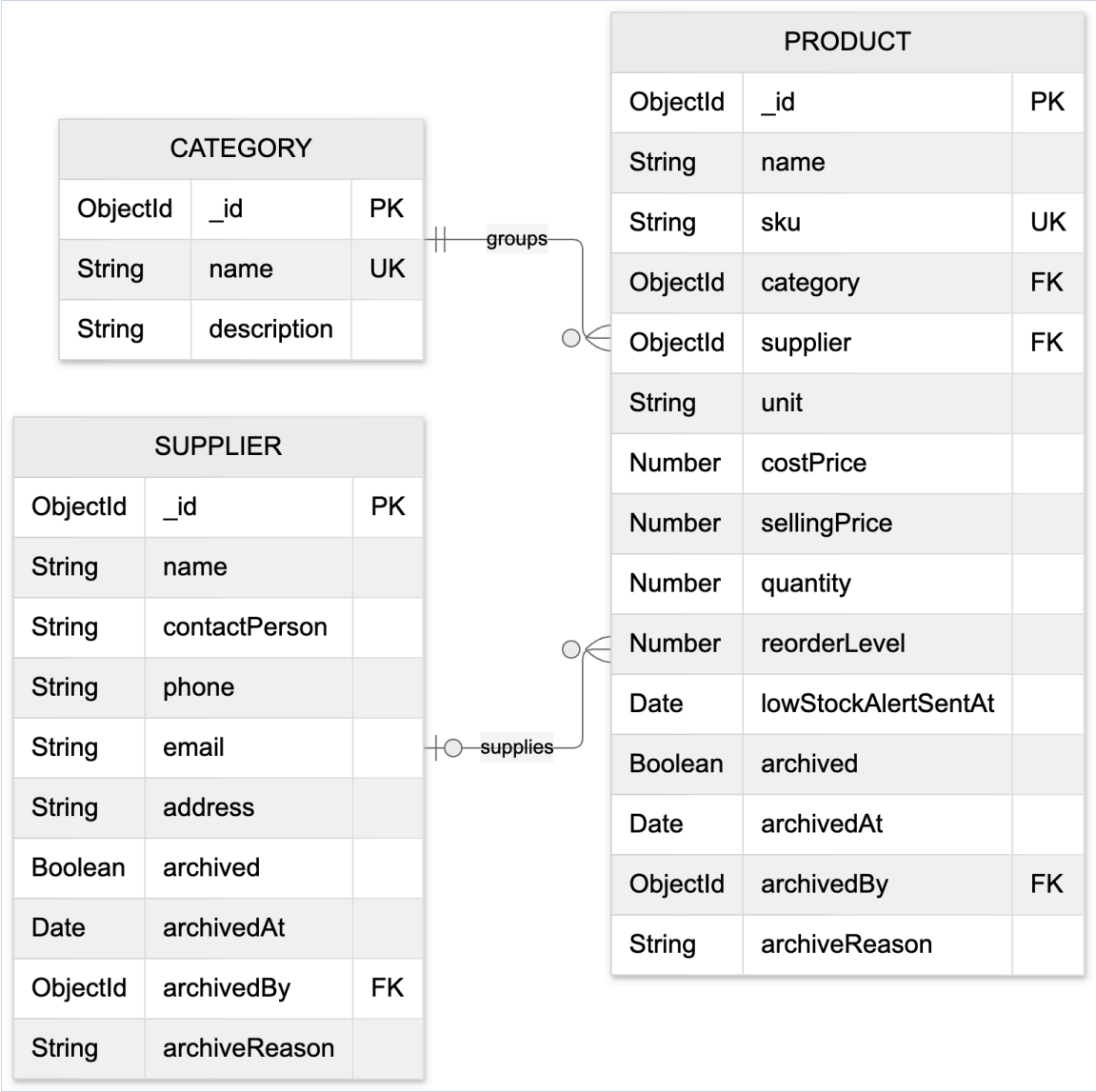


Figure 3. *Entity relationship diagram, part 1: product catalog with archive fields*

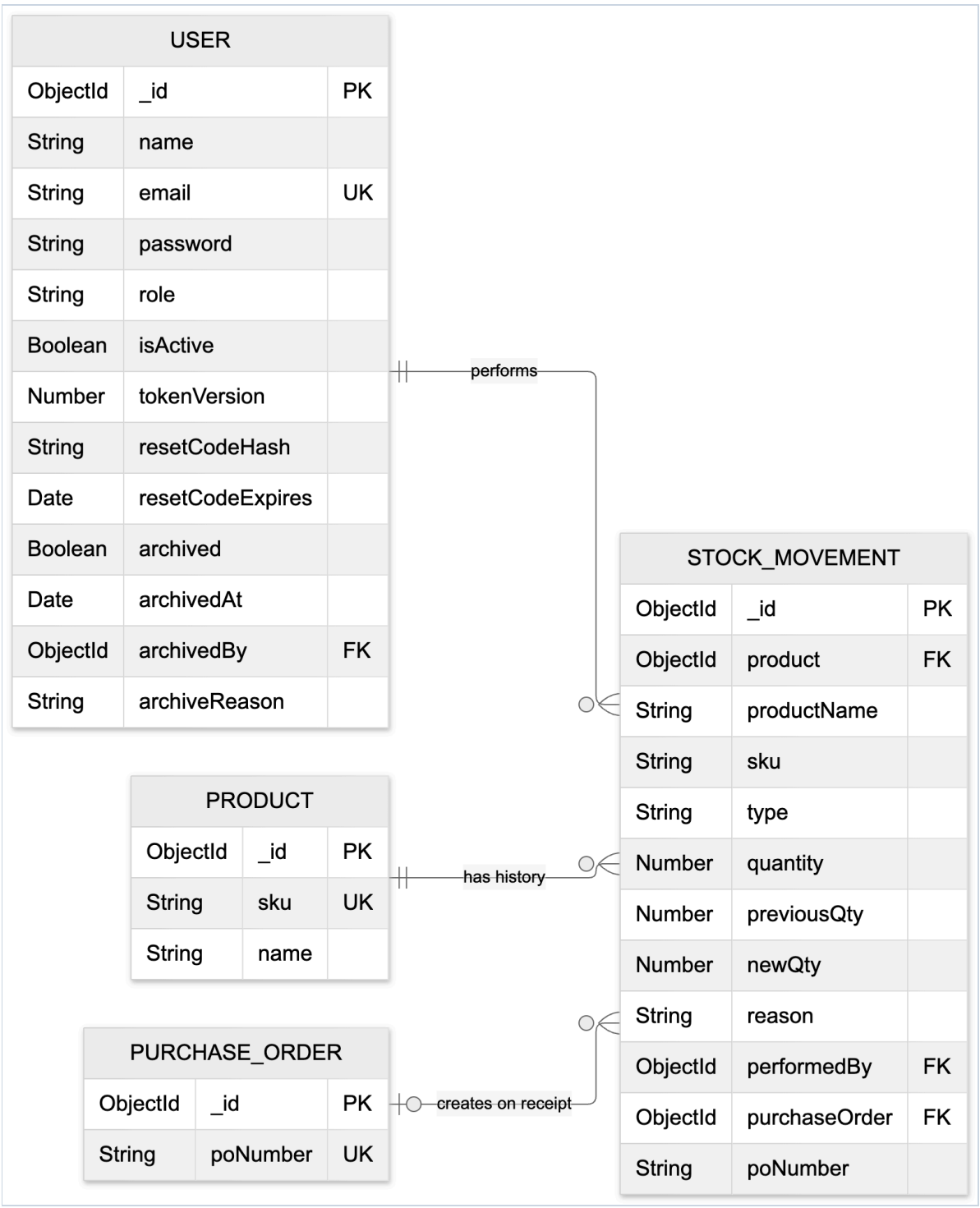


Figure 4. *Entity relationship diagram, part 2: users and stock history*

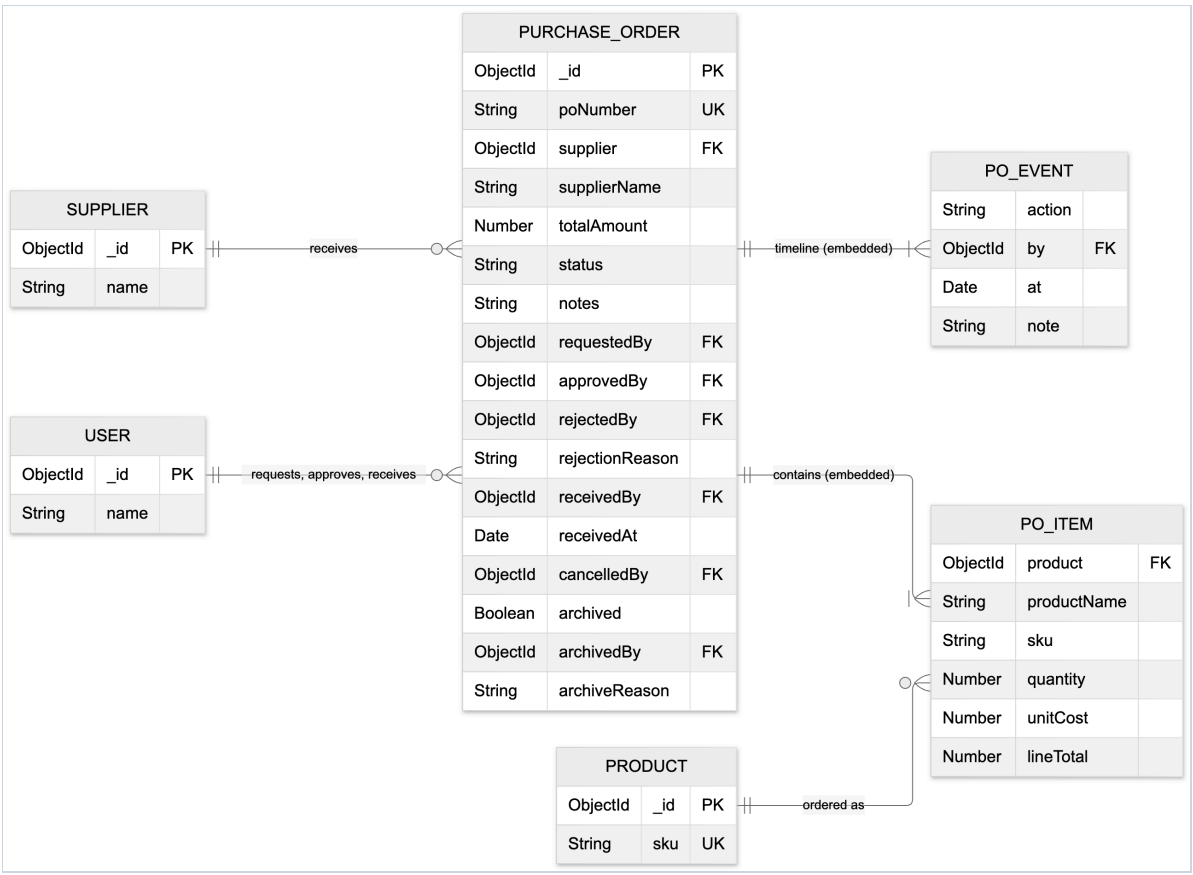


Figure 5. *Entity relationship diagram, part 3: purchase orders*

- **Category to Product (one to many):** every product belongs to exactly one category.
- **Supplier to Product (zero or one to many):** a product may have a usual supplier.
- **Supplier to PurchaseOrder (one to many):** every PO is addressed to one supplier.
- **User to PurchaseOrder:** a PO records who requested, approved, rejected, received, cancelled and archived it.
- **PurchaseOrder to items and events (embedded, one to many):** items list the products, quantities and unit costs; events form the timeline.
- **Product and User to StockMovement (one to many):** each movement belongs to a product and records who performed it; movements created by receiving a PO also reference that PO.

Table 3. *Archive fields shared by users, suppliers, products and purchaseorders*

Field	Type	Constraints	Description
archived	Boolean	Default false; indexed with archivedAt	Hidden from active lists when true
archivedAt	Date	Set when archived, cleared on restore	When it was archived
archivedBy	ObjectId	References users	Admin who archived it
archiveReason	String	Required when archiving, max 300	Why it was archived
archiveLog	Array	{ action, by, at, reason }	Every archive and restore

Table 4. *users collection*

Field	Type	Constraints	Description
_id	ObjectId	Primary key	Unique identifier
name	String	Required, max 60 characters	Full name
email	String	Required, unique, lowercase, valid format	Login and recovery email
password	String	Required, min 8, hidden from queries	bcrypt hash of the password
role	String	admin or staff, default staff	Determines permissions
isActive	Boolean	Default true	Inactive users cannot log in
passwordChangedAt	Date	Set on password change	Shown on the profile page
tokenVersion	Number	Default 0, hidden from responses	Bumped to sign out old sessions
resetCodeHash	String	bcrypt hash, hidden	Hash of the 6-digit recovery code

Field	Type	Constraints	Description
resetCodeExpires	Date	15 minutes after issue, hidden	Code expiry
resetCodeAttempts	Number	Max 5, hidden	Wrong guesses for the current code
resetRequestedAt	Date	Hidden	For the one-minute cooldown
archive fields	—	See Table 3	Archived users cannot log in

Table 5. *categories collection*

Field	Type	Constraints	Description
_id	ObjectId	Primary key	Unique identifier
name	String	Required, unique (case-insensitive), max 50	Category name, e.g. Plumbing
description	String	Optional, max 300 characters	Short description

Table 6. *suppliers collection*

Field	Type	Constraints	Description
_id	ObjectId	Primary key	Unique identifier
name	String	Required, max 80 characters	Supplier business name
contactPerson	String	Optional, max 60	Name of the contact person
phone	String	Optional, valid phone format	Contact number
email	String	Optional, valid email format	Contact email address
address	String	Optional, max 200 characters	Business address
archive fields	—	See Table 3	Blocked while it has open POs

Table 7. *products collection*

Field	Type	Constraints	Description
_id	ObjectId	Primary key	Unique identifier
name	String	Required, max 100 characters	Product name
sku	String	Required, unique, uppercase	Stock keeping unit code
category	ObjectId	Required, references categories	Product category
supplier	ObjectId	Optional, references suppliers	Usual supplier
unit	String	pcs, box, kg, L or pack	Unit of measure
costPrice, sellingPrice	Number	≥ 0	Prices per unit (₱)
quantity	Number	Integer ≥ 0; changed only by movements	Units in stock
reorderLevel	Number	Integer ≥ 0, default 10	Low-stock threshold
lowStockAlertSentAt	Date	Null until alerted; cleared on restock	Prevents repeated alerts
image	String	Optional URL or data URL ≤ 1 MB	Product photo
stockStatus, stockValue	Virtual	Computed, not stored	Status and quantity × cost
archive fields	—	See Table 3	Blocked while on open POs

Table 8. *stockmovements collection*

Field	Type	Constraints	Description
_id	ObjectId	Primary key	Unique identifier

Field	Type	Constraints	Description
product	ObjectId	Required, references products	Product that changed
productName, sku	String	Required	Snapshot kept after changes
type	String	IN or OUT	Direction of the movement
quantity	Number	Integer > 0	Units moved
previousQty, newQty	Number	Required	Stock before and after
reason	String	Purchase, Return, Sale, Damaged, Expired, Adjustment, Other	Why the stock changed
note	String	Optional, max 300	Free-text remark
performedBy	ObjectId	Required, references users	User who recorded it
purchaseOrder, poNumber	ObjectId, String	Set when created by receiving a PO	Link to the purchase order

Table 9. *purchaseorders collection*

Field	Type	Constraints	Description
_id	ObjectId	Primary key	Unique identifier
poNumber	String	Required, unique, e.g. PO-2026-0042	Human-readable number
supplier, supplierName	ObjectId, String	Required; active supplier	Supplier and name snapshot
items	Array	1 to 50 items; each product once	Embedded PO items (Table 10)
totalAmount	Number	Computed from the items	PO total (₱)
notes	String	Optional, max 500	Delivery notes or quotation number

Field	Type	Constraints	Description
status	String	Requested, Approved, Rejected, Received, Cancelled	Workflow state
requestedBy	ObjectId	Required, references users	Who created the request
approvedBy, approvedAt	ObjectId, Date	Set on approval	Admin who approved
rejectedBy, rejectedAt, rejectionReason	ObjectId, Date, String	Reason required	Admin who rejected and why
receivedBy, receivedAt	ObjectId, Date	Set on receipt	Who received the delivery
cancelledBy, cancelledAt, cancellationReason	ObjectId, Date, String	Requester or admin	Who cancelled and why
history	Array	{ action, by, at, note }	Timeline events
archive fields	—	Closed POs only (see Table 3)	Archive state

Table 10. *Embedded purchase order item*

Field	Type	Constraints	Description
product	ObjectId	Required, active product	Ordered product
productName, sku, unit	String	Snapshots	Readable after product changes
quantity	Number	Integer ≥ 1	Units ordered
unitCost	Number	≥ 0	Cost per unit (₱)
lineTotal	Number	quantity × unitCost	Computed line amount

Table 11. *counters collection*

Field	Type	Constraints	Description
_id	String	Sequence name, e.g. purchaseOrder	Which sequence
seq	Number	Incremented atomically with $inc	Last number issued

User Interface Design

The interface uses a consistent indigo and slate palette, rounded cards and color-coded badges: green, amber and red for stock status, and amber, blue, green, red and gray for the purchase order states. The screenshots were captured at 1440 pixels (desktop) and 390 pixels (mobile) with the presentation data for Santos Family Hardware.

Sign-in and Account Management

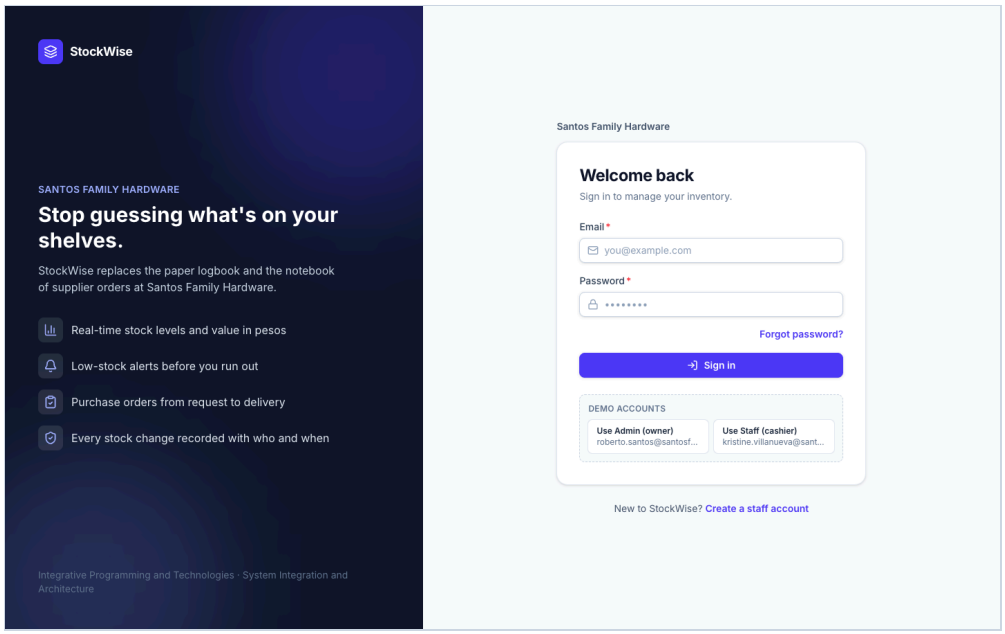


Figure 6. *Login page with the store name and Forgot password link*

The login page shows Santos Family Hardware as text, validates the form inline and links to password recovery. Demo-account buttons fill in an owner or a cashier account.

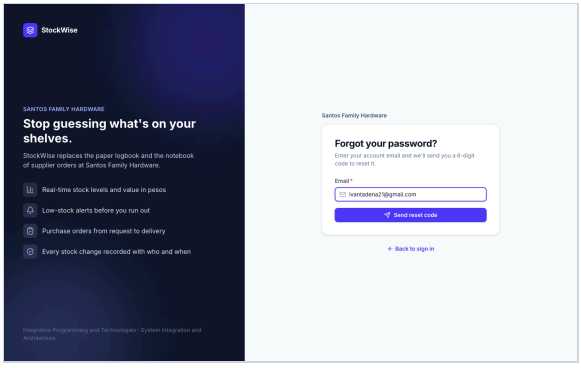


Figure 7. *Forgot password: request a code*

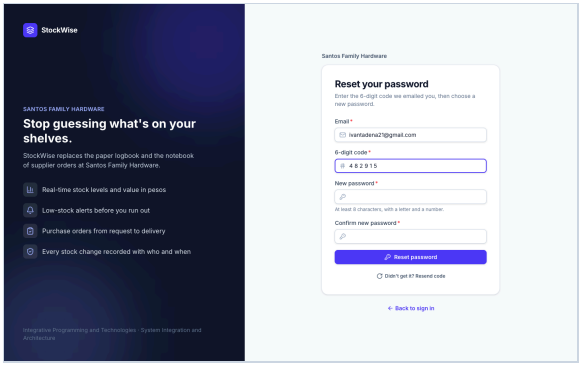


Figure 8. *Reset password with the code*

Password recovery has two steps: the user enters their email, then types the six-digit code from the email and a new password. The first page gives the same answer for every email, so it cannot be used to discover accounts.

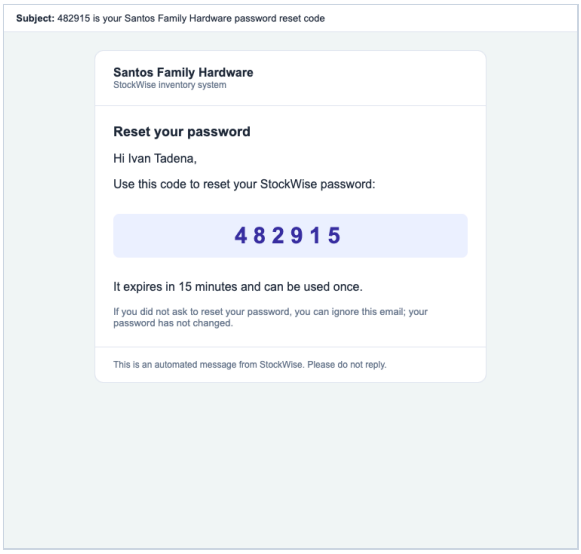


Figure 9. *Password reset email*

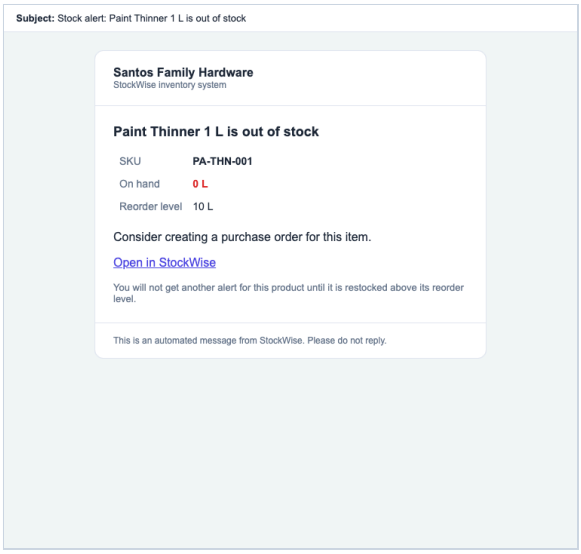


Figure 10. *Low-stock alert email*

Both emails are sent through Gmail SMTP with the store's name. The reset code expires in 15 minutes; the stock alert links to the product and is sent once until the product is restocked.

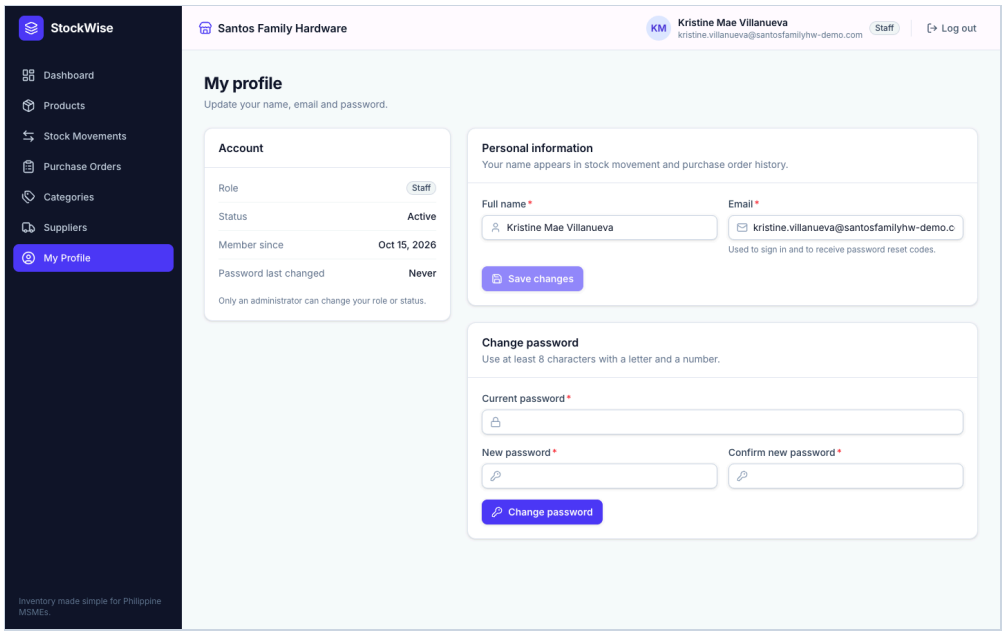


Figure 11. *My profile*

Every user can change their name, email and password. Changing the password signs out the user's other devices. Role and status are shown but only an admin can change them.

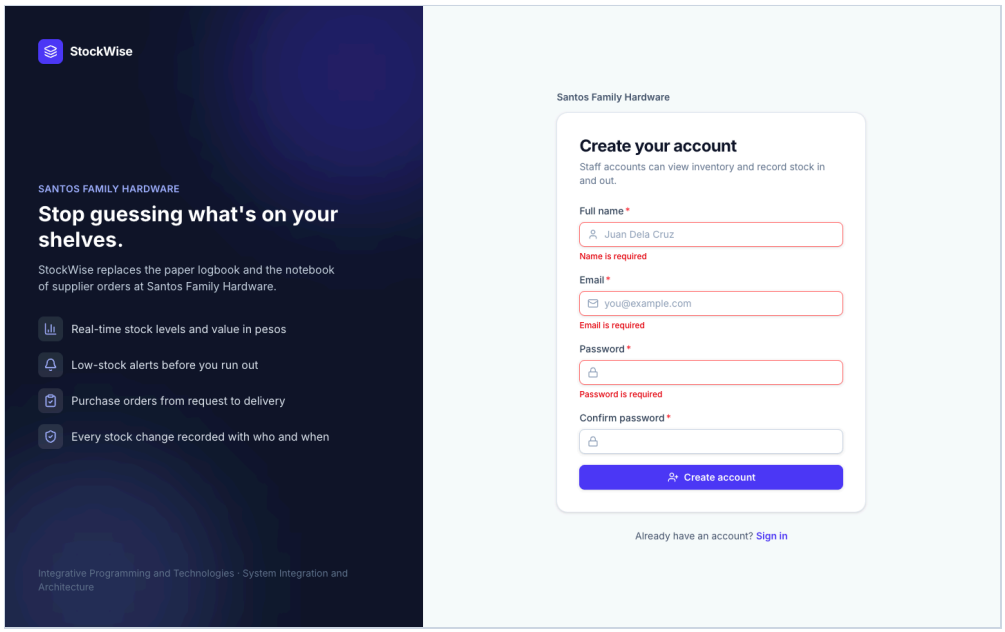


Figure 12. *Registration form with validation errors*

Public registration creates staff accounts only. Each invalid field shows its own message before any request is sent.

Dashboard and Inventory

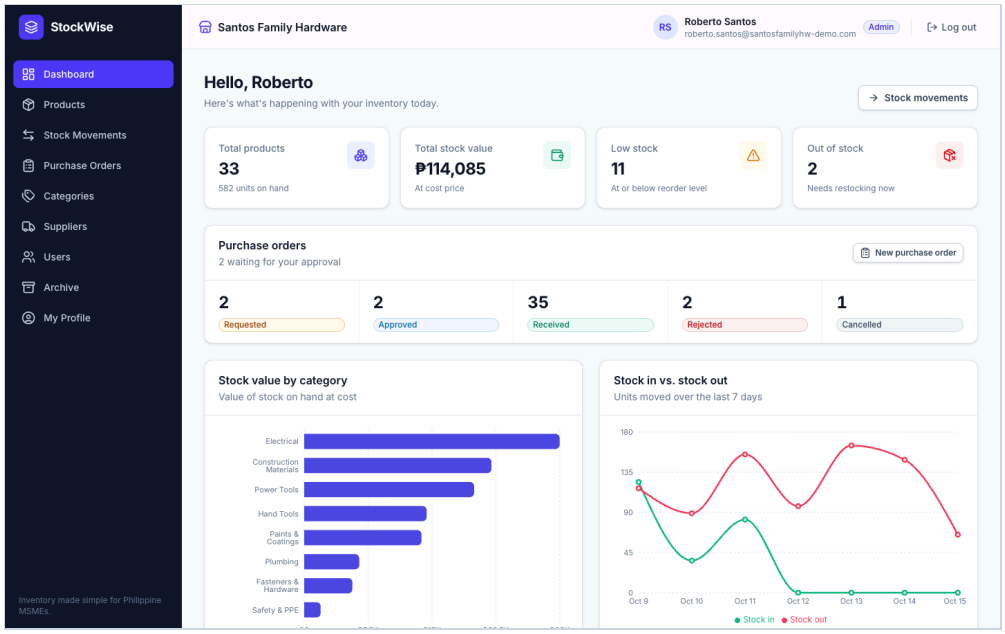


Figure 13. *Admin dashboard with purchase order counts*

The dashboard shows the number of products, stock value, low-stock and out-of-stock counts, and a tile for each purchase order status that opens the filtered list. For admins it also says how many requests are waiting for approval.

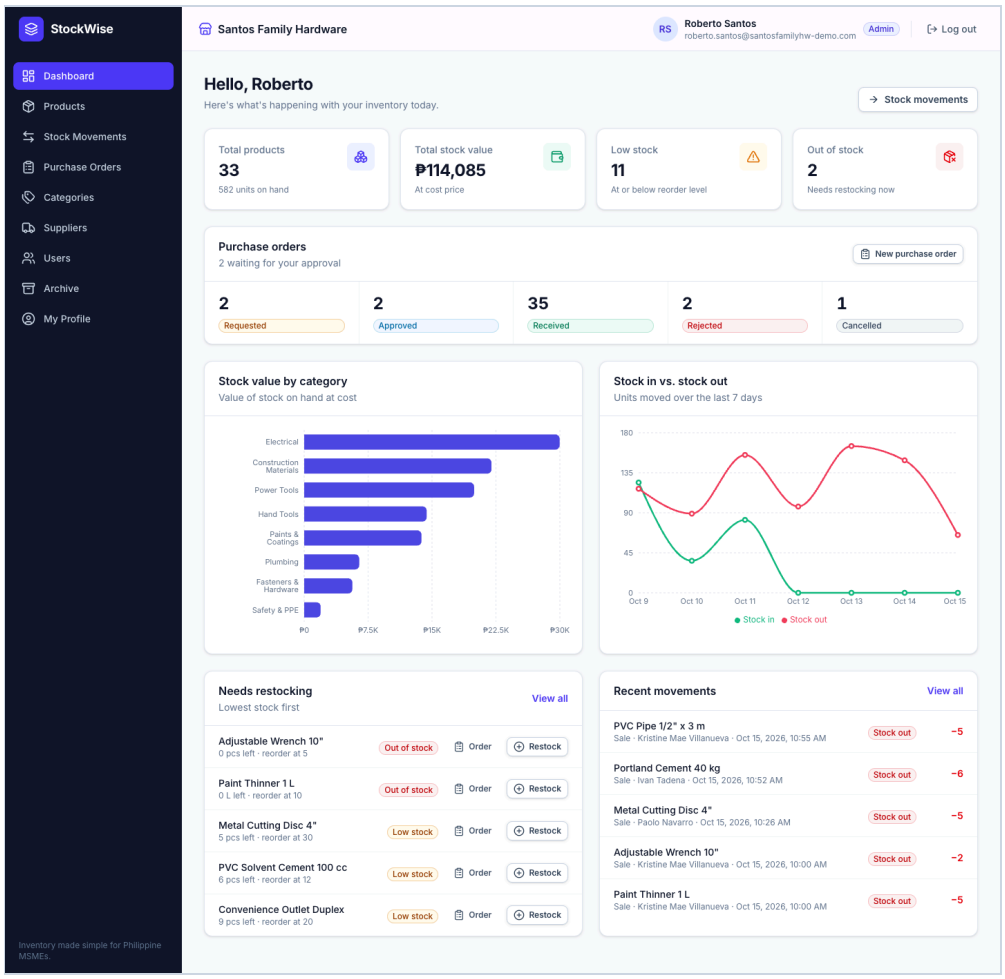


Figure 14. *Dashboard charts, restock list and recent movements*

The charts show stock value by category and units moved in and out over seven days. Each item in "Needs restocking" has an Order shortcut that starts a purchase order and a Restock button for a direct stock-in.

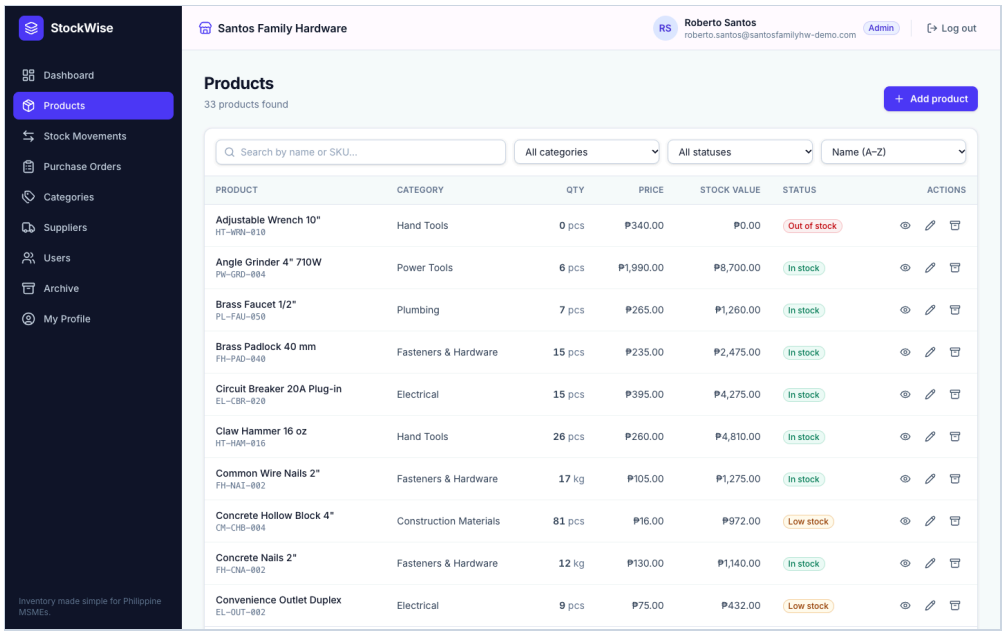


Figure 15. *Products list*

Products can be searched by name or SKU, filtered by category and stock status, sorted and paginated. Admins see view, edit and archive actions; archived products never appear here.

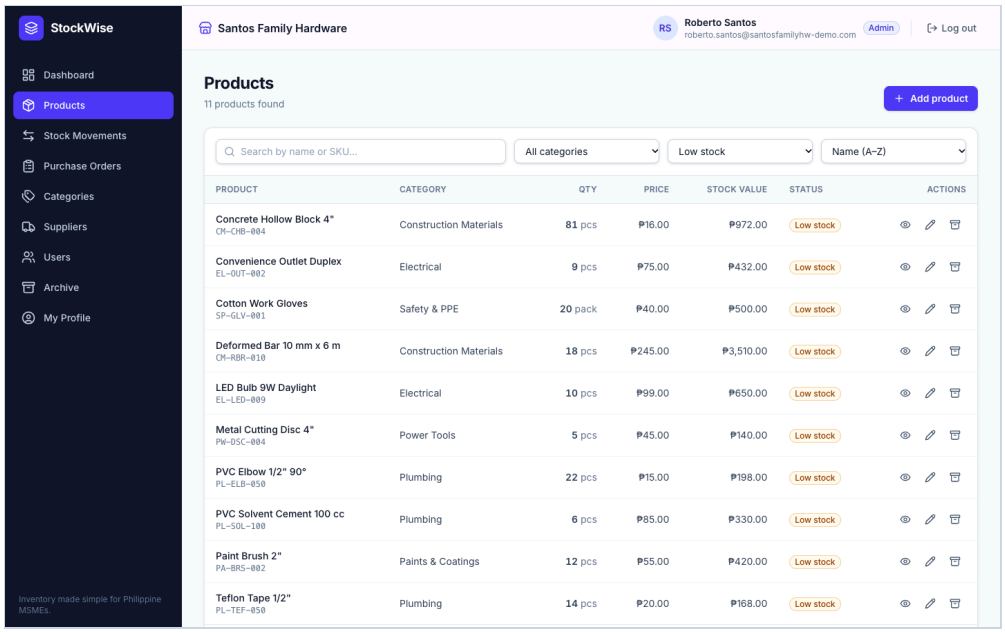


Figure 16. *Products filtered by low stock*

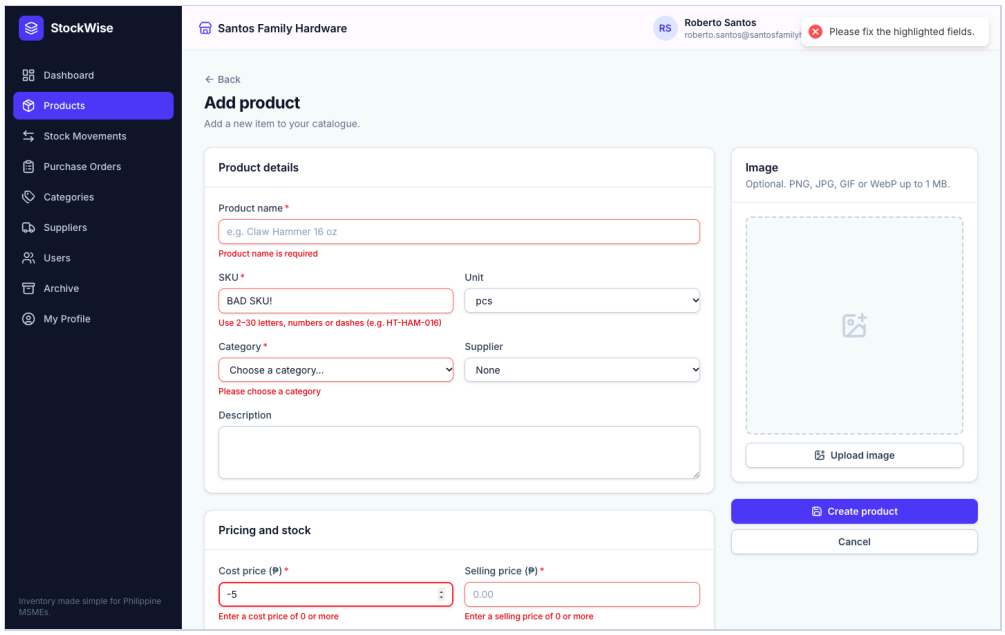


Figure 17. *Product form with validation errors*

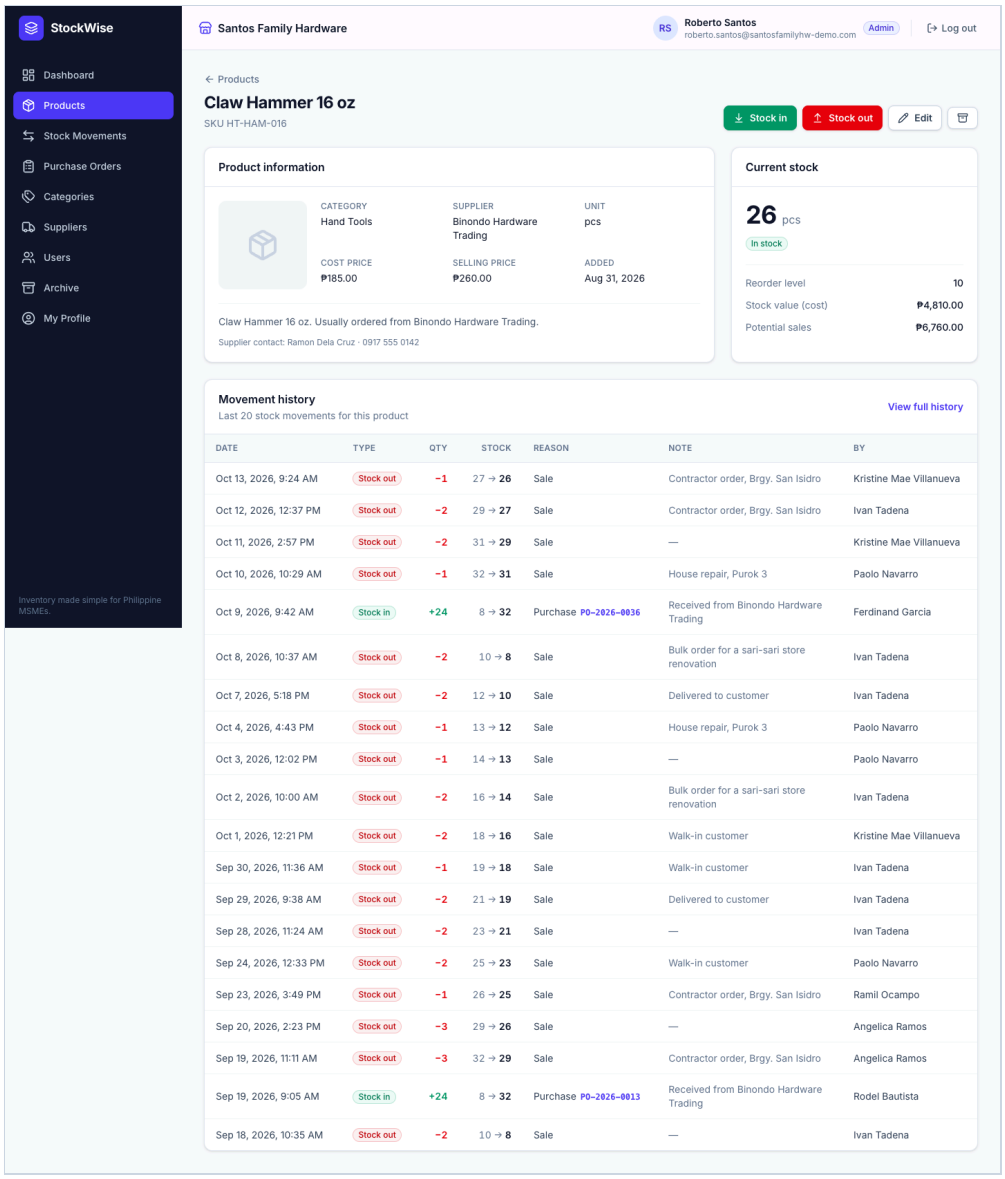


Figure 18. *Product details with movement history*

Movements created by receiving a purchase order show the PO number as a link.

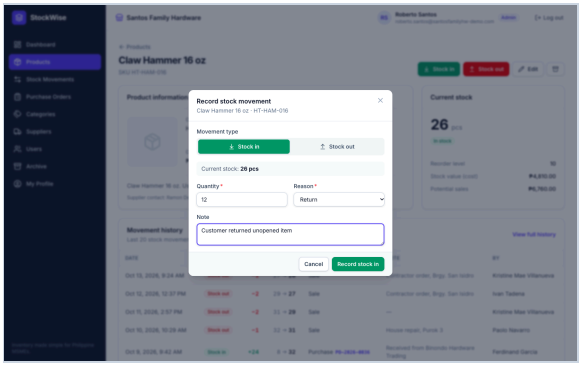


Figure 19. *Stock movement form*

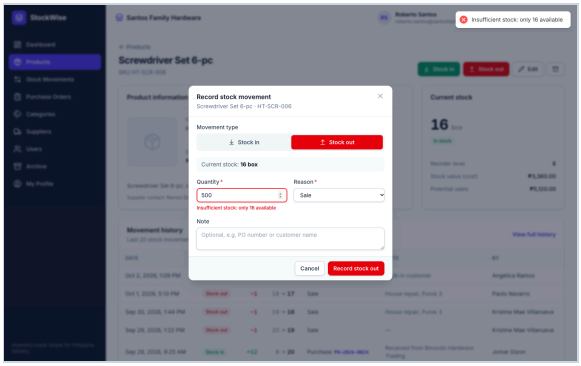


Figure 20. *Insufficient stock (409)*

Stock in and out are recorded in a dialog. A stock-out larger than the stock is rejected by the API; a sale that leaves a product low shows a notice that the admins were alerted.

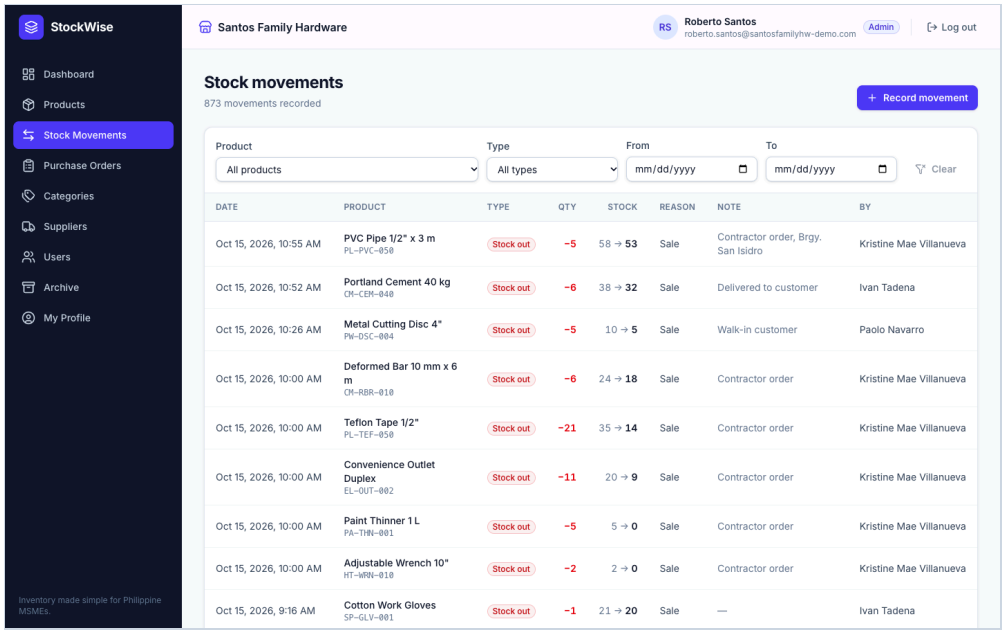


Figure 21. *Stock movements history*

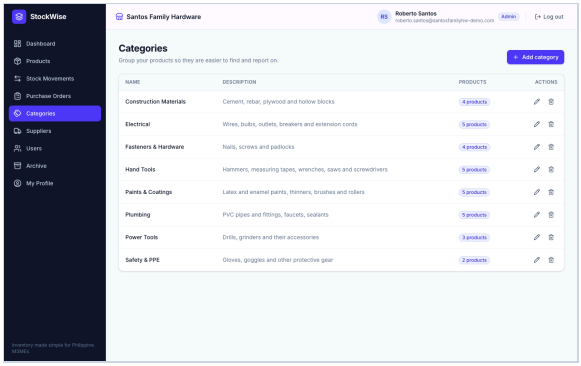


Figure 22. *Categories*

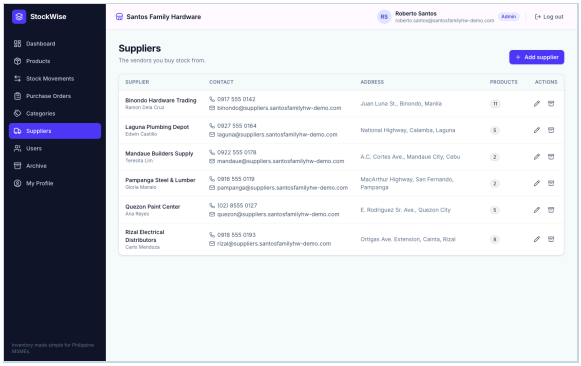


Figure 23. *Suppliers*

Purchase Order Screens

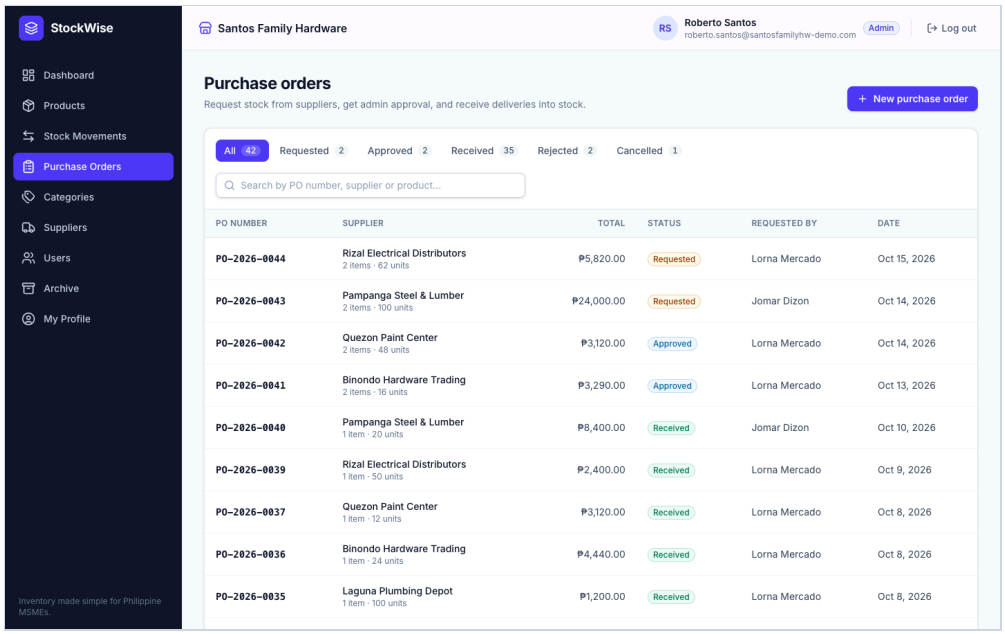


Figure 24. *Purchase order list with status filters*

Tabs filter the list by status and show how many POs are in each step. POs can be searched by number, supplier or product.

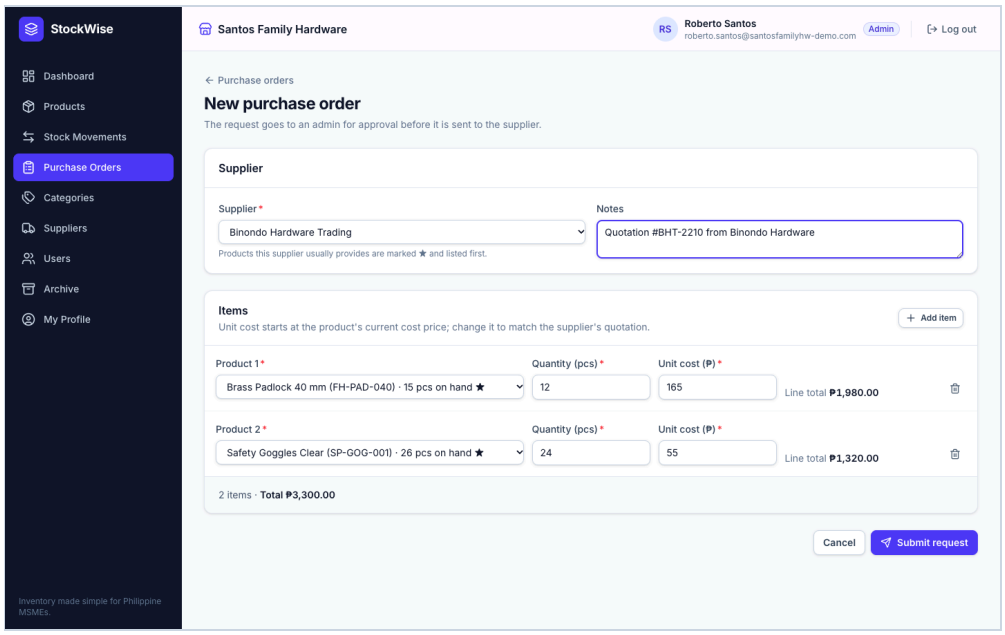


Figure 25. *New purchase order*

The form offers only active suppliers and products, lists the chosen supplier's products first, pre-fills the unit cost from the product's cost price, computes line totals and the total, and refuses the same product twice.

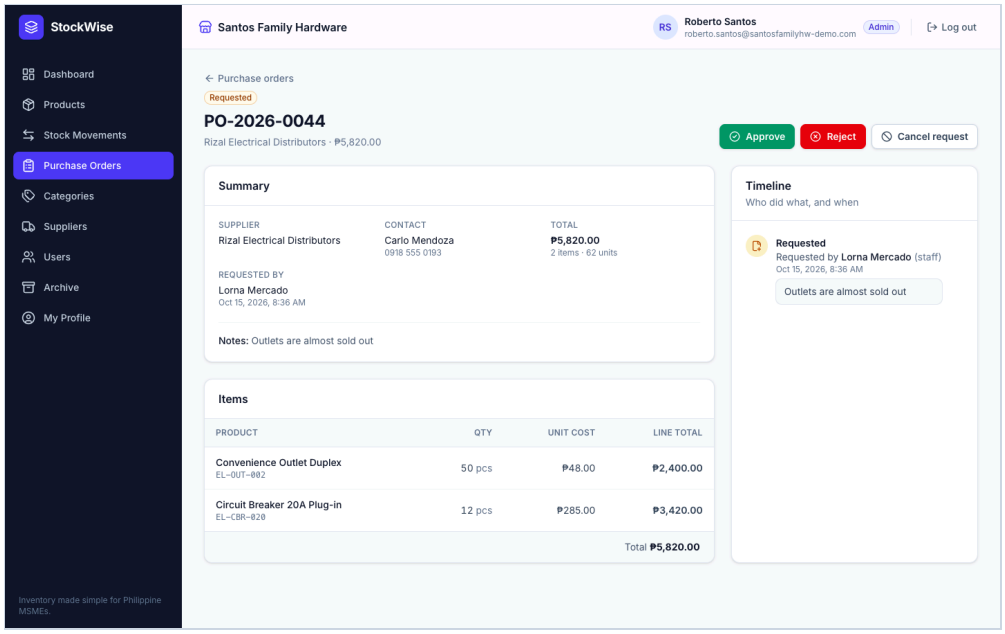


Figure 26. *Requested purchase order as seen by an admin*

An admin can approve or reject a Requested PO; the requester or an admin can cancel it. The timeline on the right records who did what and when.

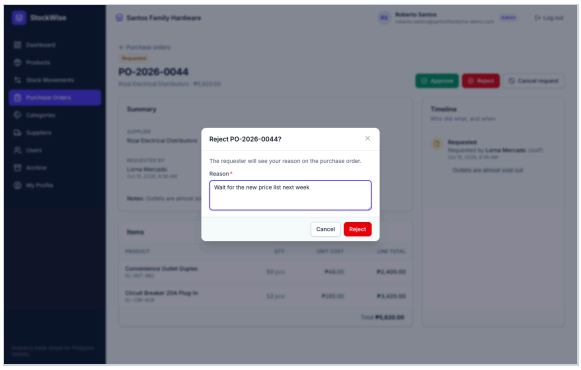


Figure 27. *Rejecting requires a reason*

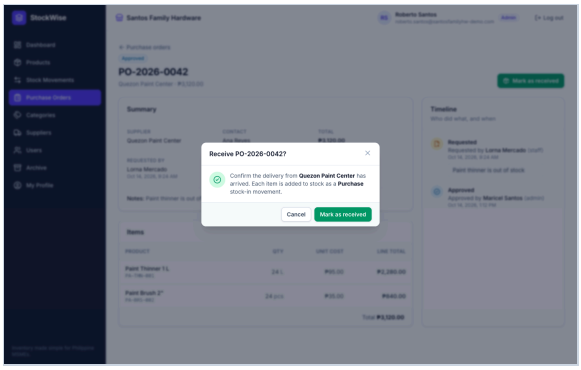


Figure 28. *Receiving an approved PO*

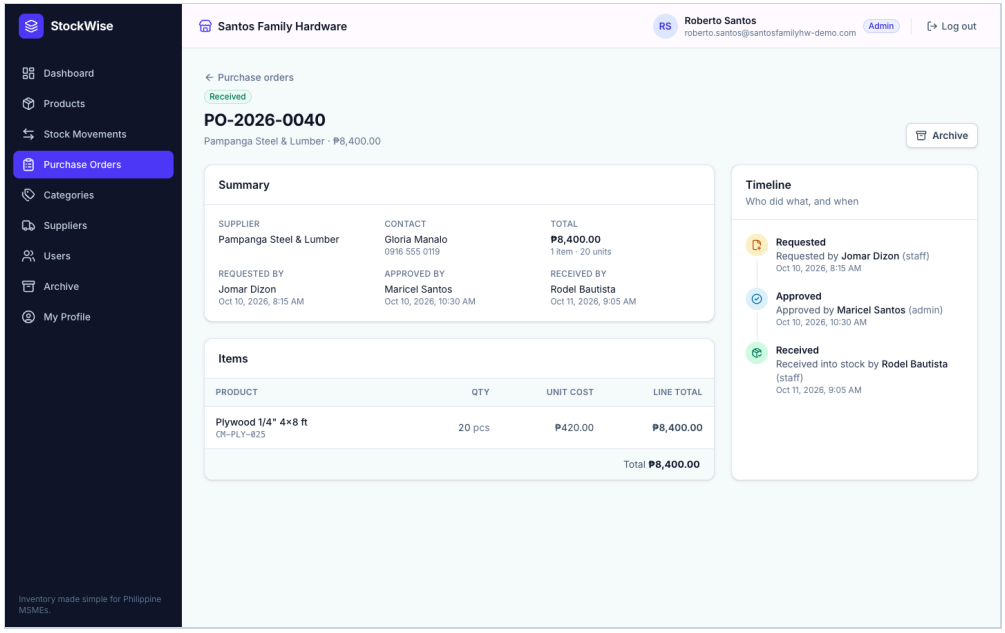


Figure 29. *Received purchase order with its timeline*

Receiving is the completed state: each item was added to stock as a Purchase stock-in movement. Closed POs can be archived by an admin.

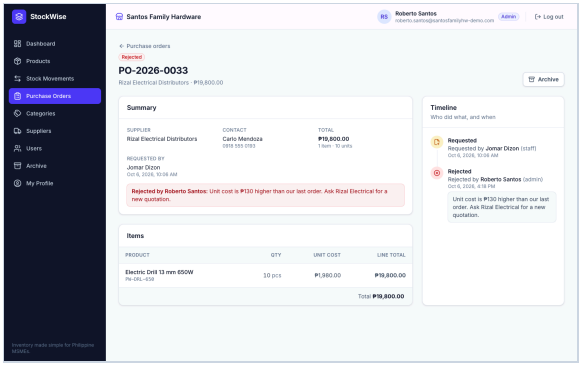


Figure 30. *Rejected PO with the reason*

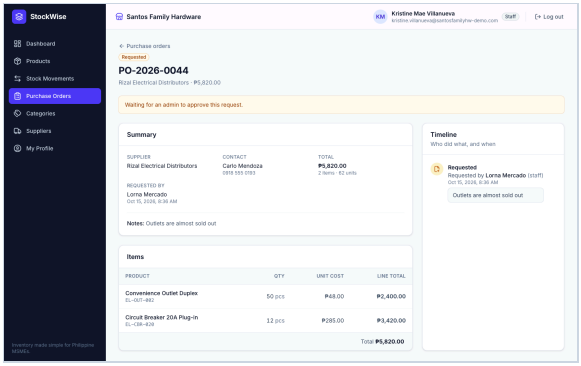


Figure 31. *Requested PO as seen by staff*

User Management and Archive Screens

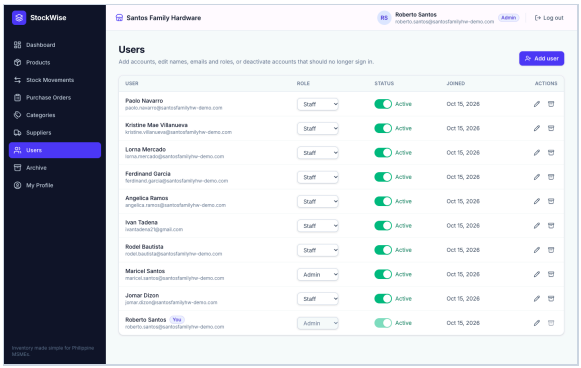


Figure 32. *Users (admin only)*

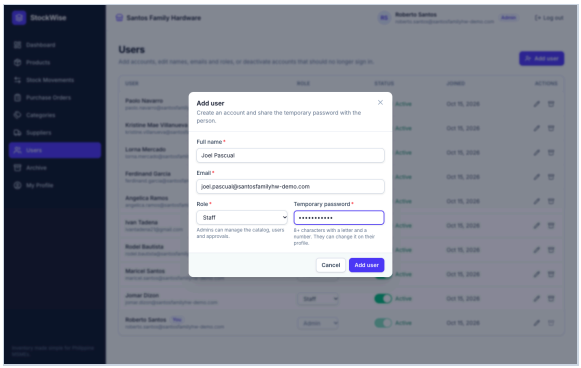


Figure 33. *Add user with a role*

Admins add users with a temporary password, edit names, emails and roles, switch accounts on or off, and archive accounts of people who left. The controls for the admin's own account are disabled.

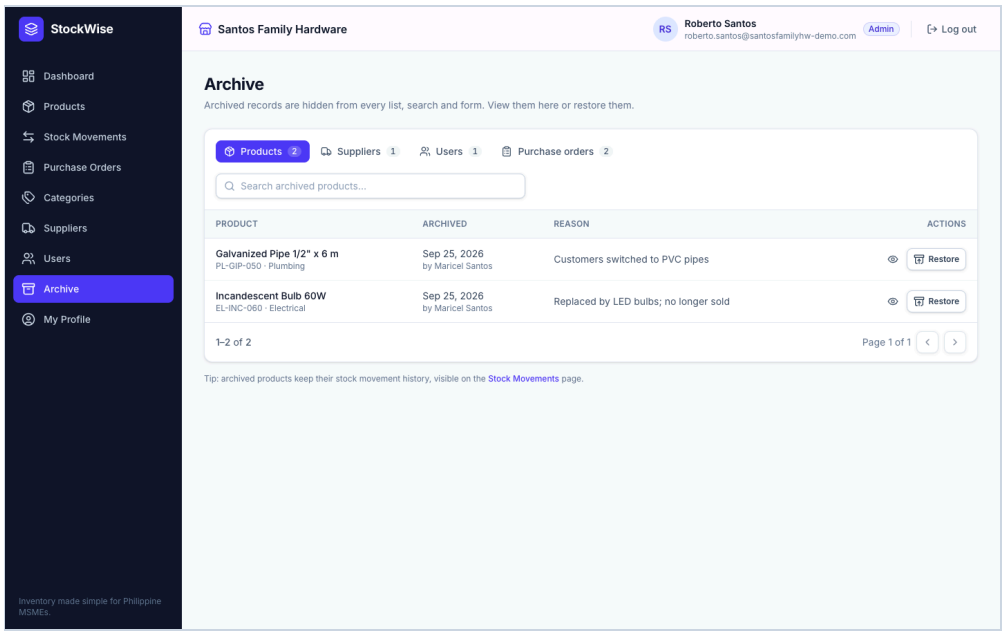


Figure 34. *Archive page*

The Archive page has one tab per record type and shows when, by whom and why each record was archived. Admins can view and restore records.

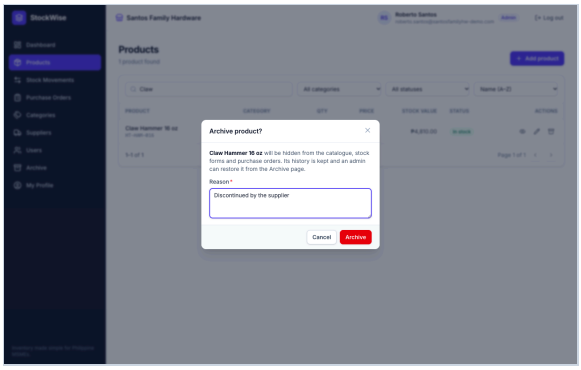


Figure 35. *Archiving asks for a reason*

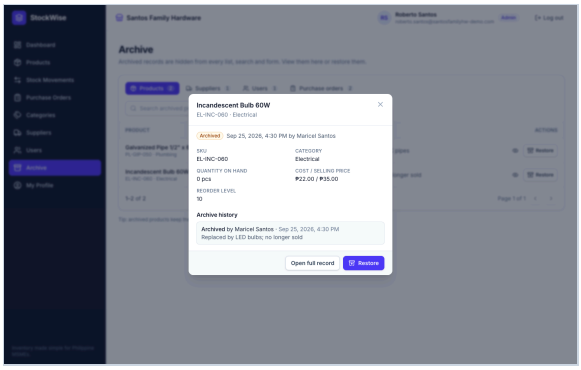


Figure 36. *Viewing an archived product*

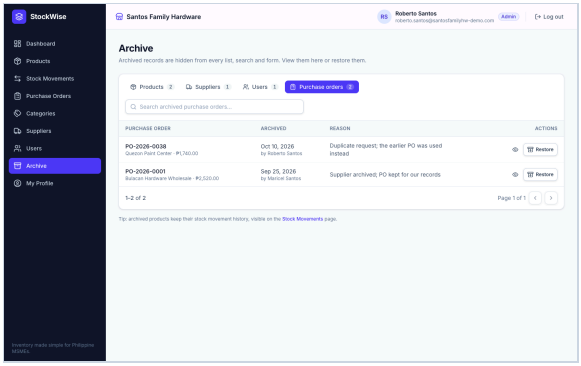


Figure 37. *Archived purchase orders*

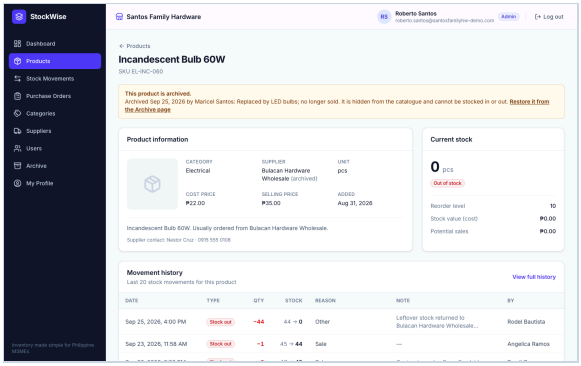


Figure 38. *Archived product banner*

An archived product can still be opened from old history, but it is marked as archived and cannot be stocked in or out.

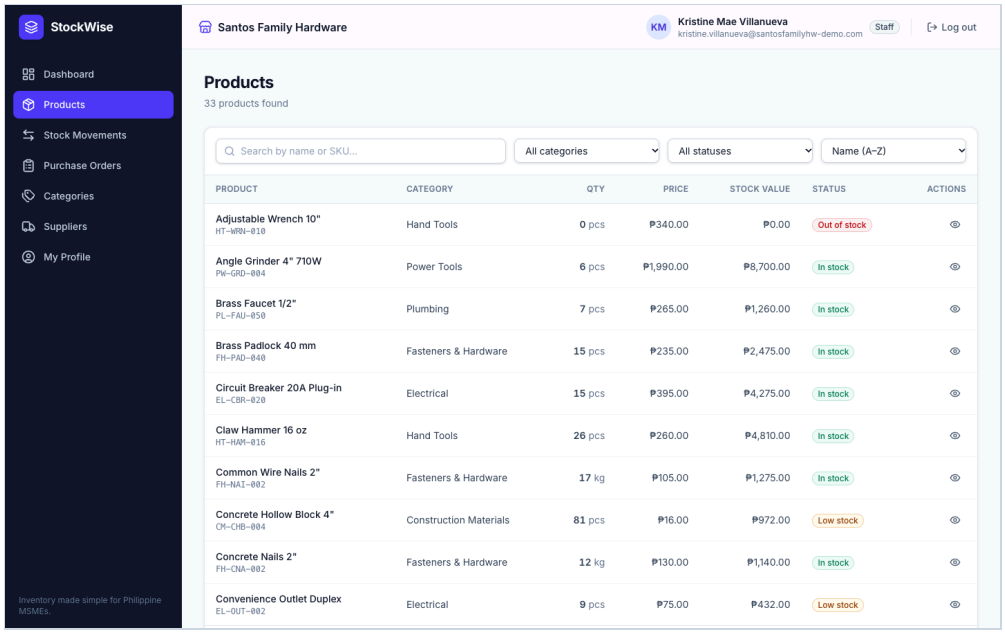


Figure 39. *Products page as seen by staff*

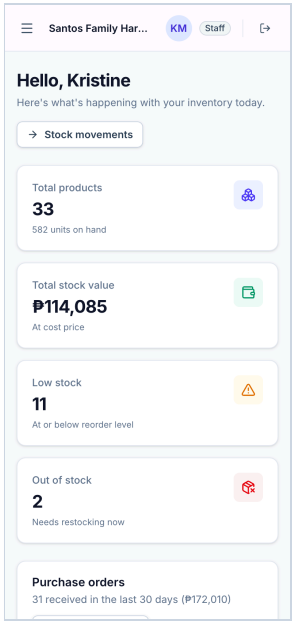


Figure 40. *Dashboard*

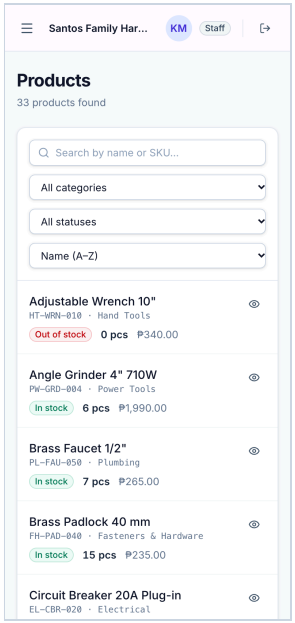


Figure 41. *Products*

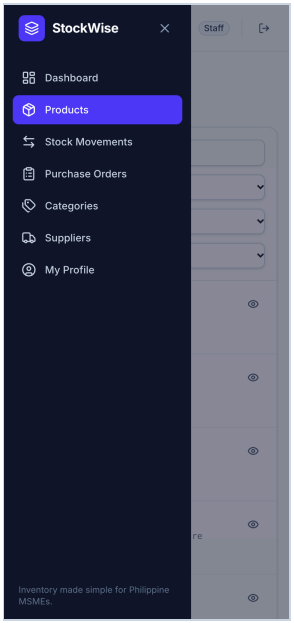


Figure 42. *Menu*

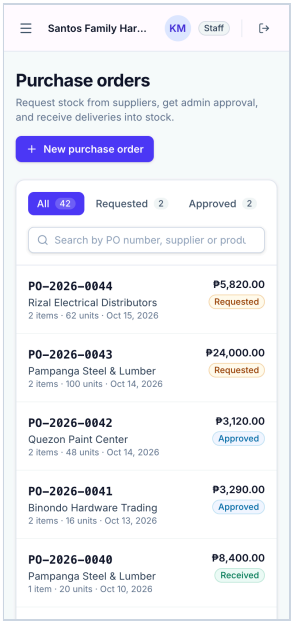


Figure 43. *Purchase orders*

On phones the sidebar becomes a drawer, tables become cards and the status tabs scroll sideways.

Feature List

Authentication and Account Management

- Staff self-registration and login with JWTs that expire after one day; bcrypt-hashed passwords never returned by the API.
- My Profile page: edit name and email, change password (current password required); other sessions are signed out.
- Password recovery: a six-digit code emailed through Gmail SMTP, stored as a bcrypt hash, valid for 15 minutes and five guesses, with a one-minute cooldown per account and per-IP rate limits. In development, the code is printed in the backend console if email is not configured.
- Deactivated and archived users cannot log in, and their existing sessions stop working immediately.

User Management

- Admins add users with a role and a temporary password, and edit their name, email and role.
- Activate or deactivate accounts, and archive accounts of people who have left.
- Admins cannot demote, deactivate or archive themselves; role checks are enforced by both the API and the UI.

Purchase Orders

- Staff and admins create PO requests with a supplier, items, quantities and unit costs; totals are computed by the server.
- Admins approve, or reject with a reason; the requester or an admin cancels a Requested PO.
- Marking an approved PO as Received records one Purchase stock-in movement per item using the shared atomic stock update.
- PO list with status tabs and counts, search and pagination; details page with summary, items and a timeline.

- Purchase order counts and an Order shortcut on the dashboard; PO numbers link from the movement history.

Archive

- Archive replaces delete for products, suppliers and users; admins can also archive received, rejected or cancelled POs.
- A reason is required; archivedAt, archivedBy and an archive log keep the history.
- Archived records never appear in active lists, searches, dropdowns, stock forms or PO forms.
- Archiving is blocked for products and suppliers with open POs and for the admin's own account.
- Archive page with one tab per type, where admins view and restore records.

Inventory, Dashboard and Alerts

- Products with search, filters, sorting, pagination, images and status badges; categories and suppliers with product counts.
- Stock in and out with reasons, atomic updates that never allow negative stock, and a filterable history.
- Dashboard statistics, charts, restock list, recent movements and PO counts.
- Email alert to the admins when a product becomes low or out of stock, once per product until it is restocked.

Data and Quality Tools

- npm run seed: presentation data for Santos Family Hardware from a deterministic 45-day simulation (10 users, 33 products, more than 30 received POs, sales, demo POs in every state, archived records).
- npm run check-data: prints the record counts, checks that every stock level equals its movements and that received POs match their stock-ins, and flags test or dummy data.
- Automated tests run only on a separate stockwise_test database and refuse to run anywhere else.

API Endpoints

Table 12. *REST API endpoints added or changed for System Integration and Architecture*

Method	Endpoint	Access	Purpose
PATCH	/api/auth/me	Logged in	Edit own name and email
PATCH	/api/auth/me/password	Logged in	Change own password
POST	/api/auth/forgot-password	Public, limited	Email a 6-digit reset code
POST	/api/auth/reset-password	Public, limited	Set a new password with the code
POST	/api/users	Admin	Add a user with a role
PATCH	/api/users/:id	Admin	Edit name, email, role or status
GET	/api/purchase-orders	Logged in	List with status, search, counts
POST	/api/purchase-orders	Logged in	Create a PO request
GET	/api/purchase-orders/:id	Logged in	Details and timeline
PATCH	/api/purchase-orders/:id/approve	Admin	Requested → Approved
PATCH	/api/purchase-orders/:id/reject	Admin	Requested → Rejected (reason)
PATCH	/api/purchase-orders/:id/cancel	Requester or admin	Requested → Cancelled
PATCH	/api/purchase-orders/:id/receive	Logged in	Approved → Received, stock IN
PATCH	/api/{products|suppliers|users|purchase-orders}/:id/archive	Admin	Archive with a reason
PATCH	/api/{products|suppliers|users|purchase-orders}/:id/restore	Admin	Restore an archived record
GET	/api/archive/:type	Admin	Archived records of one type

Method	Endpoint	Access	Purpose
GET	/api/dashboard/summary	Logged in	Now includes PO counts

The endpoints from the first version (authentication, categories, suppliers, products, stock movements and dashboard) are unchanged, except that DELETE for products, suppliers and users was replaced by archive. All responses use { success, data } or { success: false, message, errors }.

Summary and Future Work

Summary

StockWise now covers the full inventory and purchasing cycle of Santos Family Hardware: stock is tracked accurately, supplier orders go through a controlled request, approval and receiving process, and every change can be traced to a person and a time. The system integrates a React client, an Express REST API, MongoDB Atlas and Gmail SMTP, and it adds account management, password recovery, user management, purchase orders and an archive module to the Integrative Programming version without breaking any existing feature.

The code stays modular: models with a shared archive plugin, controllers, routes, validators and services on the server, and reusable components and hooks on the client. ESLint and Prettier enforce one coding style.

Testing and Verification

- **API smoke test:** 115 automated checks, including profile, password change, password recovery with the emailed code, user management, the full purchase order workflow with double-approval and double-receipt protection, the archive rules, low-stock alerts, and the original checks for CRUD, insufficient stock, invalid IDs, roles and tokens.
- **Browser end-to-end test:** 49 headless-Chrome steps for both roles, including password recovery, the profile page, adding and editing users, creating, approving, receiving and cancelling purchase orders, archiving and restoring, archived-user login, the low-stock alert and the 375-pixel mobile layout.
- **Data check:** the presentation seed is generated into a throwaway database and verified with npm run check-data on every test run.
- **Isolation:** one command (node run-tests.mjs) seeds the stockwise_test database, starts the test servers and runs everything; the tests refuse to run against any database whose name does not end in _test.

Future Work

- **Partial deliveries and supplier invoices**, with the received quantity per item and an updated cost price.

- **Purchase order PDFs** emailed to the supplier after approval.
- **Barcode scanning** with the phone camera to find products and receive deliveries faster.
- **Multi-branch support** with separate stock per branch and transfers between branches.
- **Sales and POS integration** so that each sale records a stock-out automatically with receipt details.
- **Reports** in CSV and PDF for inventory value, movement history, purchases per supplier and low stock.

Sources

Frameworks, Libraries and Services

1. Node.js. (n.d.). *Node.js documentation*. https://nodejs.org/
2. Express. (n.d.). *Express: Fast, unopinionated, minimalist web framework for Node.js*. https://expressjs.com/
3. MongoDB, Inc. (n.d.). *MongoDB Atlas*. https://www.mongodb.com/atlas
4. Mongoose. (n.d.). *Mongoose ODM documentation*. https://mongoosejs.com/
5. Meta Open Source. (n.d.). *React*. https://react.dev/
6. Vite. (n.d.). *Vite: Next generation frontend tooling*. https://vite.dev/
7. Tailwind Labs. (n.d.). *Tailwind CSS*. https://tailwindcss.com/
8. Remix Software. (n.d.). *React Router*. https://reactrouter.com/
9. Axios. (n.d.). *Axios: Promise based HTTP client*. https://axios-http.com/
10. Recharts. (n.d.). *Recharts: A composable charting library built on React components*. https://recharts.org/
11. react-hot-toast. (n.d.). *Smoking hot React notifications*. https://react-hot-toast.com/
12. Lucide. (n.d.). *Lucide icons*. https://lucide.dev/
13. Auth0. (n.d.). *jsonwebtoken*. https://github.com/auth0/node-jsonwebtoken
14. bcrypt.js. (n.d.). *Optimized bcrypt in JavaScript*. https://github.com/dcodeIO/bcrypt.js
15. express-validator. (n.d.). *express-validator documentation*. https://express-validator.github.io/
16. Helmet. (n.d.). *Helmet: Secure Express apps with HTTP headers*. https://helmetjs.github.io/
17. express-rate-limit. (n.d.). *Basic rate-limiting middleware for Express*. https://github.com/express-rate-limit/express-rate-limit
18. Nodemailer. (n.d.). *Nodemailer: Send emails from Node.js*. https://nodemailer.com/
19. Google. (n.d.). *Sign in with app passwords*. Gmail Help. https://support.google.com/accounts/answer/185833
20. Google. (n.d.). *Send email from a printer, scanner, or app (SMTP settings)*. Google Workspace Admin Help. https://support.google.com/a/answer/176600

21. dotenv. (n.d.). *Loads environment variables from a .env file*. https://github.com/motdotla/dotenv
22. Vercel. (n.d.). *Express on Vercel*. https://vercel.com/docs/frameworks/backend/express
23. Puppeteer. (n.d.). *Puppeteer documentation*. https://pptr.dev/
24. Mermaid. (n.d.). *Diagramming and charting tool*. https://mermaid.js.org/
25. docx. (n.d.). *Generate .docx files with JavaScript*. https://docx.js.org/
26. OpenJS Foundation. (n.d.). *ESLint*. https://eslint.org/
27. Prettier. (n.d.). *Prettier: An opinionated code formatter*. https://prettier.io/

References

28. Department of Trade and Industry. (n.d.). *MSME statistics*. https://www.dti.gov.ph/resources/msme-statistics/
29. Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc7519
30. Eastlake, D., & Panitz, A. (1999). *Reserved top level DNS names* (RFC 2606). Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc2606
31. MDN Web Docs. (n.d.). *HTTP response status codes*. https://developer.mozilla.org/en-US/docs/Web/HTTP/Status
32. OWASP Foundation. (n.d.). *Password storage cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
33. OWASP Foundation. (n.d.). *Forgot password cheat sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Forgot_Password_Cheat_Sheet.html
34. MongoDB, Inc. (n.d.). *Atomicity and transactions*. https://www.mongodb.com/docs/manual/core/write-operations-atomicity/